The Model Knows the Textbook but Not Your Class: Curriculum Knowledge Graph Constraints for Slide Generation in Two Vocational-Education Courses

Suxing Liu[a,b,*], Byungwon Min[a], Zhijian Zheng[c], Jiahao Wang[d]

[a]*Department of IT Engineering, Mokwon University, Daejeon, 35349, Republic of Korea*
[b]*School of Digital Media Technology, Jiangxi Arts & Ceramics Technology Institute, 538 Xinchang West Road, Zhushan District, Jingdezhen, 333001, Jiangxi, China*
[c]*University of Alabama, Tuscaloosa, 35487, AL, USA*
[d]*School of Computer Sciences, Universiti Sains Malaysia, Minden Heights, Penang, 11800, Malaysia*

Abstract

Evaluating generated teaching material raises a question benchmarks do not: does it satisfy its course specification? We make that computable. Course-level knowledge—what a session must cover, what this cohort mislearns, what must not appear yet—is compiled into a contract that is also the acceptance criterion, so requirement and check cannot drift. We ask whether the form of this representation, not the information it carries, changes how reliably a model uses it.

Across one queue module, three backbones and a second course, the same misconception text as flat passages is covered in five of nine runs (0.556 ± 0.527); inside the contract's slot, in all nine (1.000 ± 0.000). Coverage rises from 0.61–0.70 to 0.98 where the backbone follows structural instructions, and a length-matched placebo excludes prompt length. The misconception slot prices at 0.889, the forbidden list at 0.083, and the factors interact rather than add. Reliability is conditional on the backbone; ordering stays a boundary, not a result. Seven measurement defects and a blind review matching the automatic ranking (ICC(2,3) 0.92) complete the account.

Keywords:
curriculum knowledge graph, constrained generation, misconception modelling, educational knowledge engineering

*Corresponding author.
Email address: bentondoucet@gmail.com (Suxing Liu)

1. Introduction

1.1. A concrete failure

Asked for the slides of a session on queues, a general-purpose large language model (LLM) returns a well-organised deck with accurate terminology and runnable code. A teacher skimming it nods; a teacher reading it closely frowns. Three failures recur.

- The objective says students must *implement* enqueue and dequeue on a sequential queue using a Python list. The deck never shows how to write it: it covers the definition, the false-overflow problem and the circular queue as the remedy—everything except the step the syllabus requires students to perform.
- The session carries a misconception that more than 60% of this cohort reproduces in the final exam: "a Python `list` is a linked list underneath." The model does not mention it, because it is not a textbook topic but a property of *these* students.
- The model moves the circular queue's modular arithmetic onto the sequential-queue slide. Framed as "broadening the view," it inverts the prerequisite order: students meet `% maxsize` before they know what `rear` is.

None of the three is a generation-capability problem. The model can write this content; it does not know what *this course* requires, where *this cohort* errs, and how *this institution* sequences its sessions.

The size of that gap is measurable, and one number previews the paper. The misconception above, handed to the model as a flat passage, is covered in five runs out of nine; placed in the contract's misconception slot it is covered in all nine (§5.4). The text is identical in both cases, so what the contract adds is not information—it is the structural position that makes the same text reliably usable. §5.5 crosses the two and shows they are not additive: the contract is what makes the text land.

1.2. Thesis

The bottleneck has shifted from *knowledge acquisition* to the representation and application of course-level knowledge. A general model holds disciplinary knowledge (what a queue is, why FIFO matters); slide production needs *situated* knowledge, of three kinds. It is *cohort-specific*—this year's students confuse the empty and full conditions above 60%, a fact that

lives in teacher experience and assignment statistics and in no public corpus; *institution-specific*—our department merges "linked queue" and "introducing BFS" into one session, a scheduling decision that varies by institution; and *landing-point specific*—this course requires "can write the code", not "can state the concept", which the curriculum standard sets and the textbook does not. None can be obtained from a larger model or a better prompt; all must be supplied from outside. The question is in what form, such that the model can follow it *and* the system can afterwards check whether it did.

Two claims are on offer here, and they have different lifespans. The first is an *effect claim*: a graph-compiled contract improves generated decks along named dimensions (§5.4). The second is a *scope claim*: within a curriculum graph, only some dimensions can be compiled into requirements that are simultaneously generation constraints and acceptance criteria, and the boundary between the two sets is measurable rather than a matter of taste (§1.5). The effect claim is bounded by two courses in one department and will need replication; the scope claim is not, because it is a property of the representation rather than of the syllabus. We try to keep the two apart throughout, and where a result belongs to the first we say so.

1.3. Our answer

A knowledge graph, because it satisfies four requirements at once. It is *expressive* enough for heterogeneous entities (misconceptions, skills, competence levels); `prerequisites_for` is a directed edge, so a topological sort *is* a legal teaching order; one session's subgraph *compiles* mechanically into a generation contract; and that contract is *reusable as ground truth*, so what is demanded and what is checked share one source and cannot drift. The fourth is the difference from "writing the key points into the prompt": a prompt is discarded once written and compliance cannot be recovered from it afterwards, whereas a contract is a data object readable by programs both before and after generation.

1.4. Contributions

1. **A curriculum-graph schema for vocational programming courses.** We extend the node skeleton of K12-KGraph [1] by promoting `Misconception` to a first-class node and adding two edge types: `addresses_misconception`, which generates the contract's misconception section and is scored by `misconception_coverage`; and `implements`,

which today supplies only an evidence anchor and is read by no headline metric (§7.6). The asymmetry is stated rather than glossed.

2. **Constraint-as-data: one object for the demand and for the check.** `UnitConstraints` drives both prompt construction and post-hoc validation—order, required/optional split, sibling-recap caps and forbidden-term lists are derived mechanically from the subgraph; no *session-specific* rule is hand-written (Table 1). The difference from "one schema for the prompt and one for the validator" is testable: co-located schemas let a checker defect pass invisibly, whereas a single data structure read twice gives a mistake one place to live and the ablation a price to report. We found seven measurement defects in our own pipeline, each written independently of the contract and each silently favouring the constrained condition (§7.1).
3. **A scope claim for constraint-as-data, and the method that measures its edge.** Only dimensions of a curriculum graph that compile into requirements on the *structure* of the output—presence, absence, a corrective slot—can serve at once as generation constraint and acceptance criterion. Order is representable but, in the interface tested here, not withholdable from the model, and so falls outside the method's range. The ablation that prices a component also identifies the dimension on which the representation runs out (§1.5, §5.4).
4. **A reproducible six-metric acceptance protocol**, two metrics of which are redefined for this task: prerequisite order is judged by *where a concept is named* rather than by model-reported identifiers, and coverage is matched against graph synonyms so that copying the source wording gains no advantage.
5. **An empirical study whose headline is a second moment.** Six conditions × three sessions × three resamplings (54 runs), three ablations (27 runs) and a length-matched placebo, plus a 2×2 factorial crossing the misconception text with the contract slot—the interaction is $+0.556$ and the slot alone is slightly negative—with the claim-bearing conditions re-run on a second vendor's model and both contrasts replicated on a second course (§5.5, §5.7). Misconception handling differs in the mean *and* in the variance (1.000 ± 0.000 against 0.556 ± 0.527), and the collapse of dispersion is the result, because a preparation workflow cannot absorb a coin flip. The ablation prices each block separately (Table 5): 0.889 of misconception coverage for the slot, 0.083 of out-of-graph control for the

forbidden list, and one documented side effect, the slot's 0.037 of coverage.

6. **A blind teacher review that independently reproduces the automatic ordering.** Three teachers scored ten decks on five five-point items and an overall score, yielding ICC(2,3) = 0.92; rescaled to $[0, 1]$, their misconception ratings fall within 0.09 of the automatic metric on all four conditions, and they place the difference between conditions on the same axis: misconception handling and coverage, not factual accuracy.

1.5. The scope of the claim: what compiles into a criterion

Constraint-as-data is not a claim about curriculum knowledge in general. It is a claim about the part of a syllabus that can be compiled into a hard requirement on the *structure* of an artefact, and the edge of that part is a result of this paper rather than a caveat attached to it. Three dimensions compile. *Presence*: a required concept is named, with graph synonyms counted. *Absence*: a term belonging to a later session does not appear. *Slot*: corrective content for a named misconception occupies a page of a particular kind. Each becomes an inequality over the finished deck, and each is falsifiable without a human in the loop.

A fourth dimension is representable but not compilable in the interface we tested. Order lives in the graph as a directed `prerequisites_for` edge, and a topological sort *is* a legal teaching order, so the requirement is writeable. But the prompt must list the concepts, and listing them in topological order already carries the order; an arm that deletes the explicit ordering sentence deletes a restatement of what remains. What a graph can *say* about a syllabus and what a prompt can *withhold* are therefore different sets, and this paper reports the difference instead of the membership we would have preferred. Which dimension falls on which side is measurable by the ablation of §5.4(5), which is why the scope statement predicts its own nulls: a protocol that returned a win on all four dimensions would be reporting its own bias, and seven such biases had to be removed before any result could be stated at all (§7.1).

One qualification belongs to the claim rather than to a caveat on it. The boundary is a property of the representation; its *binding force* is not. A dimension can compile into a criterion and still fail to bind on a backbone that will not execute a structural instruction—on the third backbone we tested the set of compilable dimensions is unchanged while the reliability of one of them collapses. We therefore state the scope claim for a backbone that follows structural requirements: *given such a backbone*, which dimensions

compile into criteria is fixed by the representation. Naming the executor is not a weakening; a boundary that silently assumed a compliant model would be the same error as a table of wins—a property of the experiment presented as a property of the method.

1.6. Why this belongs in Knowledge-Based Systems

The object of this paper is not "a better slide generator" but how knowledge must be represented in order to be actionable by a machine. Three recent papers in this journal set the context: Dai et al. [2] show that the *presentation format* of a graph changes what a model comprehends, Huang et al. [3] show that triples and passages are not interchangeable for long-tail facts, and Ma et al. [4] place education among the journal's active applications. We move the question from *whether knowledge improves output* to *whether the same knowledge can serve as the acceptance criterion*, and we report where it does not help—which is what Roschelle et al. [5] ask for when they argue that the criterion must be about learning rather than benchmark scores: this paper says what the criterion should be *computed from*.

2. Related work

2.1. Automatic slide generation

Document-to-slide work measures *fidelity to the input*: DOC2PPT [6] released 5,873 paired examples with four metrics; Mondal et al. [7] added end-user specifications but not layout design. Instruction-to-slide work measures *fidelity to editability*: AutoPresent [8] found programmatic generation superior to image generation because the artefacts stay re-editable, and PPTAgent [9] emits editing actions against reference decks. Our rendering choices follow this strand. Evaluation has moved from whole-deck comparison (PPTC [10]) to itemised rubrics: PresentBench [11], DeckBench [12], PPTBench [13], SlideSpawn [14] and Talk to Your Slides [15]. This is where our evaluation belongs, one step short of our question: their rubrics are authored *per instance, by an annotator reading the source material*, whereas ours must be authored *per course, by the curriculum* and be identical for every deck of the same session—otherwise the criterion moves with the artefact. Their input is a document to compress or a single instruction, and the criterion is resemblance, appeal or task completion; our input is a course specification, and the criterion is conformance. No SlidesBench protocol can express "the

deck omits the skill the syllabus requires students to implement," the most damaging failure class in a teaching context.

2.2. Educational knowledge graphs

K12-KGraph [1] is the closest knowledge-side work: nine node and fourteen relation types over the official People's Education Press textbooks, with K12-Bench (23,640 items) derived from it; even the strongest proprietary model reaches only 57% exact match—evidence that structural curriculum cognition is independent of factual recall. SciMKG [16] builds a multimodal graph over open courseware, and Ma et al. [4] apply knowledge-enhanced LLMs to course recommendation. The centre of gravity in that literature is *building* graphs; ours is *using* them. We reuse the K12-KGraph skeleton rather than replace it, but a vocational graph must additionally carry competence level (mandatory versus optional) and typical misconceptions (corrective content); K12-KGraph has an importance attribute it does not use to constrain generation, and no misconceptions at all. The retrieval side divides differently: GraphRAG [17] and the wider RAG literature [18, 19] treat a graph as a *retrieval index* that decides what the model reads; here it is an *obligation structure* that decides what the output must contain. A wrong retrieval still permits a correct answer; a violated obligation is a defect by definition.

2.3. Curriculum specifications as the criterion

Judging a teaching artefact against the course's stated intentions is the central claim of constructive alignment, whose current statement is narrower than often taken: alignment is the *validity* of the inference from outcomes to activities to assessment [20, 21]. We take the demand literally and ask what changes when the aligned object must be *computed*: the criterion becomes a program, and alignment stops being a property of a course design and becomes a property of each artefact. Encoding a unit of learning as an executable specification is an old ambition—learning-design formalisms described roles, activities and sequencing precisely enough for a conforming runtime—but the specification had to be authored by a human designer, because nothing else could write content against it. LLMs change the economics: the specification becomes cheap to satisfy but not to supply. What distinguishes our use of a schema is not the schema but its provenance and its second life: compiled per session from a course graph, then reused as

the acceptance criterion rather than only as input. The contract also inherits a limit from the engagement literature: ICAP orders engagement from passive to interactive, with quantitative outcome advantages at the higher levels [22, 23], and a deck is a passive artefact—which is why the contract carries a skill landing point and a lab rather than only a concept list, and why a deck can score 1.000 on every content metric and still be a poor lesson (§7.2).

2.4. Constrained generation

Content planning and surface realisation are kept apart in classical NLG [24]; the hallucination literature documents what happens when they collapse [25], and knowledge-enhanced generation puts an explicit knowledge source into the pipeline [26]. In the LLM era structure enters the prompt, and the form matters: the same graph as triples, paragraphs or neighbourhoods is comprehended with markedly different effectiveness [2], and triples generally beat passages on long-tail facts [3]. A stronger form acts on the decoder: grammar-constrained decoding [27] forces output into a formal language; we deliberately do not use it, because a grammar can express that a slide has a title and at most four bullets but not that this session must correct the false-overflow misconception. Content constraints require enforcement after generation or in the prompt, so acceptance must be a computation over the finished deck; where a constraint is genuinely syntactic (JSON schema, layout enumeration, bullet cap) we enforce it structurally. Reference-free metrics such as faithfulness [28] ask whether every claim is supported by the context and are silent on ours: a deck can be faithful to a passage and still omit the required skill. LLM-as-a-judge and rubric-conditioned scoring [29, 30] now track human preference well enough to serve as a primary evaluation; we use a judge only for the qualitative pass and keep the headline numbers mechanical, because a fixed criterion must yield a fixed score and a judged score cannot promise that. We neither train, nor modify decoding, nor rerank: we compile constraints into a data object, prompt with it before generation, and validate against it afterwards. The price is the risk of disobedience; §5.9 reports a real violation, and §7.1 the seven traps this permissiveness exposed.

2.5. The same premise, industrialised

Learning Commons, an initiative of the Chan Zuckerberg Initiative, ships two of the components this paper argues for: a *Knowledge Graph* of academic standards and progressions for K-12, and *Evaluators* that score AI-generated

educational content against expert-authored rubrics, with more than seventy partners including several model providers [31]. The question this paper asks is thus an engineering need with users, and the architecture chosen there is the one argued for here: a graph that holds the obligations, separated from an evaluator that checks against them. The difference is scope. A standards graph covers publicly codified expectations, where standard-to-content mapping is many-to-one and stable across districts; a vocational course's obligations are largely local—which skill *this* cohort must implement at the end of *this* session, which misconception *this* class holds—and none of that can be purchased as a dataset. The layers are complementary: a standards graph could populate the upper part of our schema, and the local layer is what a teacher adds.

3. Problem formulation

3.1. Curriculum knowledge graph

A curriculum knowledge graph is a typed directed heterogeneous graph $G = (V, E, \tau)$, where $\tau : V \to T_V$ with

$$T_V = \{\text{Course}, \text{Module}, \text{Unit}, \text{Concept}, \text{Skill}, \text{Lab}, \text{Exercise}, \text{Misconception}\};$$

each edge $e = (u, r, v) \in E$ carries a relation type $r \in T_E$ with

$$\begin{aligned} T_E = \{&\text{is_a},\ \text{prerequisites_for},\ \text{relates_to},\ \text{implements},\\ &\text{verified_by},\ \text{assessed_by},\ \text{addresses_misconception},\\ &\text{appears_in},\ \text{is_part_of},\ \text{leads_to}\}; \end{aligned}$$

and selected edges carry an `evidence` field recording the provenance of the relation (textbook page, curriculum-standard clause, error-rate statistic).

Constraint 1 (acyclicity). The subgraph induced by `prerequisites_for` must be a DAG. The graph layer enforces this: every cycle is reported as a `PREREQ_CYCLE` error and the loader refuses a graph that fails validation, so a circular prerequisite stops the pipeline before any generation happens. The compiler retains a defensive branch that falls back to declaration order, unreachable for a validated graph and pinned by two unit tests; a compiler should not crash mid-run, but a curriculum error must never be compiled silently into a prompt.

Table 1: Components of the session contract $C(u)$ and where each is read from in the graph.

Symbol	Meaning	Source
$S_u = (R_u, O_u)$	required / optional concept sets	split on `importance`; at or above "master" is required
$\prec_u$	teaching order (a total order on S_u)	topological order of `prerequisites_for` over G_u
M_u	misconceptions that must be corrected	`Misconception` nodes targeted by `addresses_misconception`
F_u	forbidden-term list	`forbidden_terms` on the Unit node
B_u	sibling-session concepts admissible as brief recap	concepts of other Units in the same Module, deduplicated
k_u	slide budget	`slide_budget` on the Unit node

3.2. Session contract

For a session $u \in V$ with $\tau(u) = \text{Unit}$, take the subgraph

$$G_u = \{\, v \in V : (v, \text{appears_in}, u) \in E \,\} \tag{1}$$

and compile it into a contract

$$C(u) = \langle S_u,\ \prec_u,\ M_u,\ F_u,\ B_u,\ k_u \rangle, \tag{2}$$

whose components are given in Table 1.

B_u and F_u must not be conflated: the former means "already covered, may be mentioned in one sentence", the latter "not taught in this course at all". Conflating them makes two metrics fight over one term, so the implementation enforces mutual exclusivity by assertion and §4.2 checks it on the graph.

Compilation adds no information; it selects and orders what the graph already contains. Fig. 2 shows the U11 subgraph and the contract compiled from it: every edge in panel (a) maps to a field in panel (b), and nothing in panel (b) is typed by hand. "Lossless" is meant in this narrow sense, with two recorded exceptions: the required/optional split applies a policy constant (the `importance` level at which a concept becomes mandatory), and Kahn's algorithm breaks ties by declaration order, so the compiled order is one of several admissible topological orders. Both are visible in $C(u)$ but they are policy, not data.

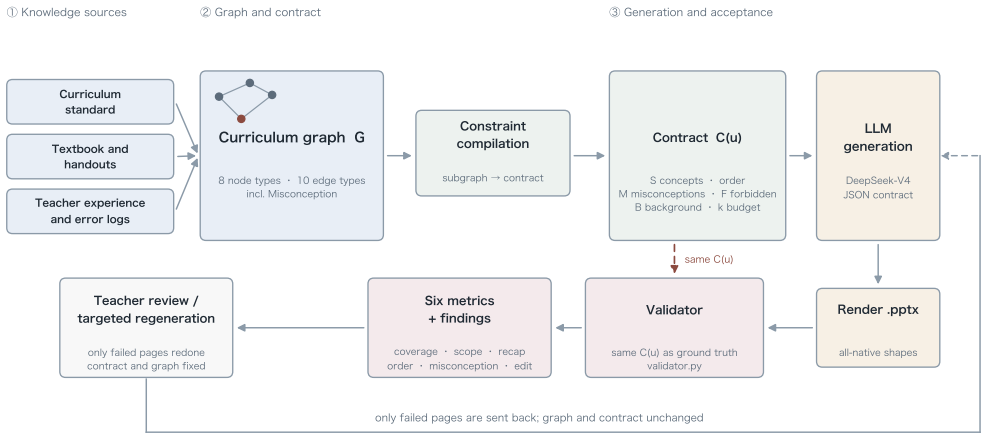


Figure 1: Framework overview. The contract $C(u)$ is read once before generation (to build the prompt) and once after (to compute the six metrics); both reads concern the same object, which is what "demand and check share one source" means in implementation terms.

3.3. Generation and acceptance

Generation is a mapping $D = f_\theta(C(u))$ where D is a sequence of slides. Acceptance is a metric vector

$$m(D, C(u), G) = \langle \text{cov},\ \text{oog},\ \text{bg},\ \text{pre},\ \text{mc},\ \text{edit} \rangle \tag{3}$$

defined in §5.3. Note that $C(u)$ appears on both sides: this is the formal statement of "demand and check share one source."

Fig. 1 shows the pipeline: graph → contract compilation → generation → rendering → validation against the same contract. One loop exists—pages that fail validation may be regenerated—and its boundary is fixed: the loop may change slides, but never the graph and never the contract. Whether to regenerate is a teacher's decision. Were validation output allowed to reshape the contract, acceptance would degrade into an objective (§7.1).

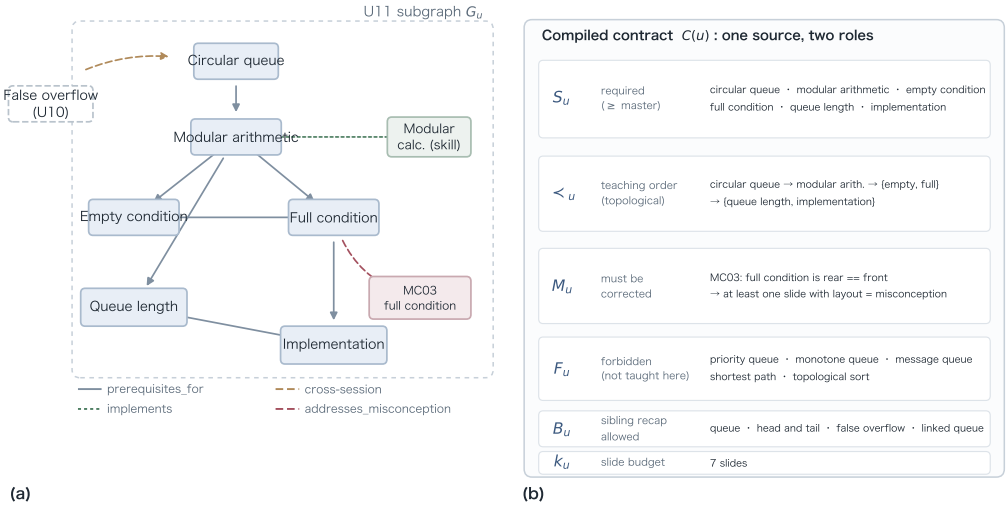


Figure 2: From subgraph to contract. (a) The actual U11 subgraph in the released graph file: six concepts and one skill, seven intra-session prerequisite edges, one cross-session prerequisite edge from U10, one `implements` edge and one `addresses_misconception` edge. (b) The contract $C(u)$ produced from it by `build_constraints`; the validator subsequently reads the same object.

4. Method

4.1. Graph construction

4.1.1. Nodes and edges

We keep the pedagogical subset of K12-KGraph's node skeleton and rename it to match course structure: its `Book`/`Chapter`/`Section` become our `Course`/`Module`/`Unit`, and its `Experiment` is rendered as `Lab`. `Misconception` is promoted to a first-class node, with attributes `statement` (the erroneous claim itself, which must be concrete and decidable), `correction`, `frequency` (low, common, high) and `evidence` (exam error rate, assignment statistics, in-class questioning).

Two edge types are added:

- `implements`: Concept or Skill → executable code fragment. In the current system it supplies the evidence anchor the prompt cites for a concept and makes the intended code artifact addressable. No headline metric reads it, and the only execution that touches it is the bounded probe of §7.6; it is not otherwise a scoring dimension.

- `addresses_misconception`: Unit, Lab or Concept → Misconception, annotated with a strategy (counter-example, code demonstration, boundary test). This edge decides which session is responsible for correcting a given misconception, preventing both duplication across sessions and orphaned misconceptions.

4.1.2. Synonyms

Every concept may carry aliases. This is not a patch for evaluation but legitimate graph content: in class the teacher says "judging full," the textbook writes "the full condition," and the code reads `(rear+1) % maxsize == front`; all three denote one entity. Without aliases the graph cannot be aligned with real teaching language. Aliases also resolve a fairness problem in evaluation, as discussed in §7.1.

4.2. The graph is checked, not merely loaded

The contract is only as trustworthy as the graph that produces it, so the graph is validated on ten checks (twelve codes; checks 5 and 10 emit two each) every time it is loaded, with no sampling step: a load that raises stops the pipeline. Checks 1–6 belong to the schema—no dangling edge, no prerequisite cycle, every knowledge point grounded in a unit, evidence on every semantic edge, three to eight concepts per unit with at least one at mastery level, no duplicate name within a module. Checks 7–10 are the ones this paper depends on, because they test whether the graph's own contract ingredients contradict one another rather than whether the file is well formed:

- $\mathbf{F_u} \cap \mathbf{R_u} = \emptyset$ and $\mathbf{F_u} \cap \mathbf{B_u} = \emptyset$. A term may not be simultaneously mandatory and forbidden (the contract would then ask for coverage and avoidance of the same thing), nor forbidden here and admissible as a sibling recap; B_u is the sibling set `build_constraints` actually emits, so the check tests the contract as shipped.
- **No orphan misconception.** An orphan can never enter any contract, and the 0.556 reported for the control conditions would then be a property of the graph rather than of generation.
- `implements` is well typed and grounded: the edge must point at an executable artefact, and its source must be taught in at least one session.

The shipped graph (34 nodes, 75 edges) returns zero findings on all ten checks, and `pytest` asserts it. Zero findings is weak evidence on its own—a

checker that never fires cannot be distinguished from one that cannot fire—so each of the twelve codes is exercised by mutation: one minimal defect is injected into a copy of the graph and we record whether the owning code responds. All twelve fire on their own mutant, and a single mutant triggers a second code (an ungrounded skill also violates check 3), which is the correct verdict rather than interference; Table S2 lists every injected defect, and `paper/graph_checks.py` is the harness.

What this buys is a weaker and more defensible claim than "the graph reasons." The graph does not infer anything: the topological order, the required set and the misconception assignment are computed by deterministic graph algorithms, and their correctness is established by the checks above, not by the model. The reasoning a knowledge-based system contributes happens earlier—in how the course was decomposed into sessions, concepts and misconceptions, and in the decision that a session's obligations are a function of that decomposition (§7.1 is explicit about which metric defects the graph can and cannot detect).

4.3. Constraint compilation

`build_constraints` turns the subgraph into a `UnitConstraints` object; every field is a function of the graph, none is entered per session: (1) *required/optional split*—sort by `importance`; those at or above "master" enter R_u, the rest O_u; (2) *topological order*—Kahn's algorithm over S_u, ties broken by declaration order, deterministic; a cycle would fall back to declaration order with a warning, but this branch is defensive only, since a cyclic graph is rejected at load time (§3.1); (3) *misconception assignment*—collect the misconception nodes attached by `addresses_misconception` edges; (4) *sibling sessions*—concept names of other Units in the same Module, minus those already in this session; (5) *evidence anchors*—collect `evidence` along incoming `prerequisites_for`, `implements` and `verified_by` edges and render them as *teacher-facing* verification material, explicitly labelled "not generated content" so that the model does not copy the provenance into the slides as body text. The compilation output is a *deterministic string*: the same graph always yields the same prompt block, which is a precondition for reproducibility.

4.4. Generation

The prompt is assembled from four segments—a fixed skeleton, the contract block, a JSON contract, and a list of hard rules—and the experimental

conditions of §5.2 replace only some of them. The assembled prompt for session U11 is rendered in full in the supplementary material (Section S7), and one raw response with its validator report in Section S8; neither is summarised away, because the prompt is part of the method.

Output is constrained to a single JSON object, each slide carrying `layout` (one of seven), `title`, `bullets`, `code`, `notes`, `concept_ids` and `background`. Forcing JSON rather than free-form layout is deliberate: layout selection is itself an instructional-design decision, and delegating it to the model makes both editability and slide budget uncontrollable. The caps on bullets and characters are conventions rather than derived constants, though not arbitrary ones: cognitive load theory predicts that dense, split-attention material imposes extraneous load competing with the schema the lesson is building [32], and the same concern returns from the learning-sciences side as an argument that benchmark scores are the wrong criterion for educational AI [5]. A four-bullet page is a rough operationalisation of both, and we treat it as a structural constraint the validator can check rather than as a claim about pedagogy.

Two engineering realities, observed rather than assumed. JSON mode is not a validity guarantee—roughly one call in ten returns a trailing comma, or blank with `finish_reason == "stop"`—so the pipeline repairs conservatively (strip trailing commas, close truncated brackets) and escalates by re-prompting with the concrete parse error only when repair fails, since blind retries reproduce the same error at the same cost. And the model emits `reasoning_tokens` on every call, which contaminates a chain-of-thought baseline because the model is already reasoning; we record the setting and provide a switch to disable it.

4.5. Rendering

`python-pptx` emits 16:9 fully native shapes with no rasterised text. `editability` is defined as the ratio of native editable shapes to all shapes; it is 1.0 for every condition here. This dimension does not discriminate between methods; it exists to exclude "screenshot-style generation," whose value for a teacher who needs to edit is zero.

For Chinese text the East-Asian typeface attribute must be set explicitly, or presentation software falls back to a Latin font and the glyphs are corrupted.

4.6. Validation

The six metrics are defined in §5.3. Three implementation choices are deliberate.

- **Chinese has no word boundaries.** The term for *queue* matches inside the term for *circular queue*. All counting therefore uses per-line net occurrences: raw count minus occurrences covered by longer terms *within the same line*. Subtracting over the whole document rather than per line is a real bug: an independent occurrence on slide 2 is otherwise cancelled by a compound term on slide 4.
- **Triple-criterion misconception detection.** Structural anchor (does the misconception slide reference a graph-linked concept), then English-identifier overlap (`rear`, `list`), then Chinese character bigram overlap (threshold 0.35). Purely lexical matching misses cases where the model paraphrases the erroneous claim in its own words; we observed a generated correction slide rejected because its phrasing differed from the graph's.
- **Order by first textual position**, see §5.3.

5. Experiments

5.1. Course and data

The target course is *Python Data Structures*, a higher-vocational programme in Artificial Intelligence Technology Application with 136 contact hours. The graph scope is Module M4, "Queues", comprising three sessions (U10, concepts and false overflow; U11, circular queue; U12, linked queue and introducing breadth-first search), with 34 nodes, 75 edges and four misconceptions. The prerequisite chain in this module is clean (queue → sequential queue → circular queue → linked queue), whereas deep-learning courses have mutually referencing concepts that easily form cycles and are therefore unsuitable as a first validation scenario.

The goal is to establish *whether and along which dimensions* graph constraints help, not to maximise a course-wide number. Three sessions cover three qualitatively different cases: concept-dense (U11 has seven required concepts), skill-landing (U10 contains a skill that must be implemented by hand), and application-introduction (U12 requires a comparative selection against prior material).

Table 2: Six experimental conditions. Prompt lengths are measured on session U11, in characters.

Condition	Course/ session	Page budget	Concept names + definitions	Topological order	Misconceptions	Forbidden terms
`full` (ours)	✓	✓	✓	✓	✓	✓
`rag_mc`	✓	✓	✓	–	✓	–
`rag`	✓	✓	✓	–	–	–
`placebo`	✓	✓	–	–	–	–
`cot`	✓	✓	–	–	–	–
`zero_shot`	✓	✓	–	–	–	–
Prompt length, U11	1997 / 1122 / 997 / 2195 / 625 / 585					

5.2. Conditions

Six conditions carry strictly increasing information, each step removing one structural element (Table 2).

- `rag` − `zero_shot` measures the contribution of *content retrieval*; `full` − `rag` measures the contribution of *structural constraints*.
- `rag_mc` is a control built for the misconception claim: the same misconception text supplied as a flat passage, without ordering or layout requirements. If `full` still wins, the gain comes from *structuring the text to the point where it constrains layout*, not from the text appearing in the prompt.
- `placebo` is a control for the alternative explanation "the prompt is simply longer": *length-matched* to `full` (in fact slightly longer), padded with generic slide-writing advice. It does not catch up (§5.4).

What `cot` actually adds, and what `rag` does not. `cot` and `zero_shot` occupy the same row of Table 2; the difference is one added rule—before writing a page, state in its `notes` where it sits in the lecture and how it connects to the previous page. That is the cheapest stand-in for chain-of-thought prompting [33, 34] and not the same thing: the model reasons about the *deck*, not about the course, so we report it as a planning baseline. The two retrieval conditions run no retriever: their passage set is the definition of every concept in the session, inlined whole—no index, no chunking, no top-k. The identifier is kept because it names the condition in the released run files, but the mechanism is *oracle passage supply* with recall 1 by construction, which makes the baselines stronger than a deployed RAG system; what the

Table 3: Metric definitions. Arrows indicate the desired direction.

Metric	Definition	Direction
`coverage_rate`	fraction of required concepts actually taught	↑
`oog_rate`	(out-of-graph IDs + forbidden-term hits) / (covered + out-of-graph)	↓
`background_rate`	characters devoted to sibling-session concepts / total characters (cap 25%)	↓
`prerequisite_violation`	violated concept pairs / all concept pairs	↓
`misconception_coverage`	misconceptions corrected / misconceptions linked to the session	↑
`editability`	native editable shapes / all shapes	↑

comparison isolates is the *form* in which correct content arrives, not retrieval versus graph. An end-to-end retrieval baseline with a corpus and a measured recall is future work.

Three ablations retain the complete contract and rule wording of `full`, deleting exactly one data block—the ordering block with its rule 3, the misconception block with rule 5, and the forbidden-term block with rule 6 (Table 5; per-metric matrix in Table S1). The design follows one rule: the rule text must be removed together with the data, since keeping "you must follow the given order" while deleting the order would force the model to guess. A unit test guards this.

5.3. Metric definitions

The six metrics are listed in Table 3.

Three definitional decisions determine the conclusions and therefore require explicit justification: (1) coverage counts body slides only—title, objectives and summary slides list every concept by definition, so counting them as "taught" lets a table of contents score full marks; every condition emits all three pages, so the exclusion is symmetric, but it is a *convention*, not a fact about teaching, and §5.3 reports what happens when it is relaxed; (2) coverage matches on name and aliases—otherwise `full`, which copies source names, enjoys a structural advantage (§7.1); (3) order is judged by first textual position—the model's `concept_ids` are self-reported, the three baselines were never told the field exists, and scoring on self-reports means a completely empty answer receives a zero violation rate.

Whether the preview-page convention is doing the work. The first decision is the one a reader should distrust, so we recomputed coverage for all 54 main runs under two relaxations of it (Supplementary Section S10). Admitting the title slide moves two runs, leaves the ranking of the six conditions unchanged (Spearman $\rho = 1.00$; $\rho = 0.94$ once objectives and summary are also admitted) and *widens* the headline contrast against `rag_mc` ($+0.283 \rightarrow +0.302$). The aliases decision points the same way and more sharply: name-only matching drops baseline coverage to 0.325–0.389 while `full` falls only to 0.963, because `full` was handed the canonical strings and copied them. Alias matching therefore *narrows* the gap—the reported $+0.283$ is the conservative number and a stricter matcher would give $+0.574$, which is why the advantage should be read as a lower bound rather than a best case. Both corrections run against the hypothesis, not for it.

5.4. Results

Each condition runs three sessions × three resamplings (nine generations), and so does each ablation arm; the ablation file also holds its own nine reruns of `full`, which is where 90 API calls come from—54 + 27 + 9. We report 54 and 27 as the experimental counts, and 90 as calls rather than as independent units of evidence. Runs are nested in three sessions, so the intervals below resample runs within a condition and the three session means are printed beside each; no mixed model is fitted, because three clusters cannot support one, and a cluster-robust standard error over three clusters would be estimated from three numbers—it would narrow the interval without earning it.

Pass rate is the share of runs with no *error-level* finding—a missing required concept (`MISSING_CONCEPT`) or a prerequisite violation (`PREREQ_VIOLATION`); out-of-graph content and a missed misconception are warnings and do not fail a run. That is why `full` passes six of nine while missing no misconception at all, and why we do not call the system "always usable": a third of its runs still need a human.

Five observations follow, ordered by importance.

(1) Coverage: 0.98 against 0.61–0.70, but the gain is not uniform across sessions. The five controls lie within 0.09 of one another (0.614–0.698), all with large dispersion (0.19–0.30); `full` reaches 0.981. Run-level intervals do not overlap—$[0.944, 1.000]$ against $[0.524, 0.878]$ for the nearest control, `rag_mc`—a difference of $+0.283$ $[+0.101, +0.471]$ with Cliff's $\delta = +0.60$: a

Table 4: Main results ($n = 9$ per condition), mean (standard deviation). Cov.: coverage; OOG: out-of-graph; Bg.: background; Ord.: prerequisite violation; Miscon.: misconception. Best value per column in bold. Pass rate is the share of runs with no error-level finding, i.e. no missing concept and no order violation; warnings do not fail a run (§5.4). Editability is 1.000 for every condition and is therefore omitted.

Condition	Cov. ↑	OOG ↓	Bg. ↓	Ord. ↓	Miscon. ↑	Pass rate
`full` **(ours)**	**0.981** (0.056)	**0.000**	0.027 (0.016)	**0.020** (0.045)	**1.000** (0.000)	67%
`rag_mc`	0.698 (0.295)	0.111 (0.220)	0.027 (0.016)	0.007 (0.022)	0.556 (0.527)	22%
`rag`	0.659 (0.248)	0.111 (0.220)	0.032 (0.014)	0.030 (0.089)	0.222 (0.441)	0%
`placebo`	0.675 (0.266)	0.167 (0.250)	0.040 (0.018)	0.096 (0.131)	0.333 (0.500)	0%
`cot`	0.614 (0.185)	0.019 (0.056)	0.040 (0.016)	0.089 (0.183)	0.389 (0.486)	0%
`zero_shot`	0.643 (0.248)	0.019 (0.056)	0.040 (0.011)	0.037 (0.089)	0.222 (0.441)	0%

randomly drawn `full` deck outscores a randomly drawn `rag_mc` deck with probability 0.80.

That interval resamples runs and so treats nine correlated draws as independent. Resampling whole sessions—three clusters, all 27 resamples enumerated—widens it to $[-0.056, +0.667]$, and the per-session contrast shows why: $+0.238$ on U11, $+0.667$ on U12, -0.056 on U10. The coverage advantage is carried by two of three sessions and is absent in the third. We give both intervals—the run-level one because it is what nine runs per condition support, the clustered one because it is the honest uncertainty—and the claim surviving both is narrower than "coverage rises from 0.61 to 0.98" (§7.2).

The decisive evidence against the length explanation is the placebo: its prompt is comparable in length to `full` (2195 against 1997 characters on U11, in fact longer) but its content is generic writing advice, and its coverage is 0.675 $[0.505, 0.833]$, indistinguishable from zero-shot.

Concept-level pairing. Two intervals over three clusters leave the coverage claim weaker than the numbers suggest, so we ask the question one level down, where the contract is actually written. Scoring every required concept on every run gives $16 \times 3 = 48$ paired observations per baseline. Against `zero_shot` the discordant cells split 14 to 1—a paired risk difference of $+0.271$ with McNemar $p = 9.8 \times 10^{-4}$; against `rag_mc`, the control that receives the same misconception text, 11 to 1 ($+0.208$, $p = 6.4 \times 10^{-3}$). All five baselines give the same sign and all sit below $p = 0.01$ (Table S5). The pairs are not independent—sixteen concepts nested in three sessions and

resampled—so we do not read those p values as population inference; what they add is that the advantage is not carried by one or two concepts. Leaving each concept out in turn moves the per-session gap by at most 0.056, and the weak session survives the same treatment: U10 is still the only session whose sign can flip, which is why the clustered interval above still includes zero.

(2) Misconception: the mean is the lesser finding; the disappearance of variance is the greater. `rag_mc` receives the same misconception text and reaches 0.556, but with interval $[0.222, 0.889]$ and standard deviation 0.527: it succeeds fully in about half of nine generations and misses entirely in the rest. `full` reaches 1.000 with degenerate interval $[1.000, 1.000]$ and zero dispersion; the difference is $+0.444$ $[+0.111, +0.778]$. Unlike coverage, this survives clustering—$+0.333$ on U10, $+0.333$ on U11, $+0.667$ on U12, so the cluster interval $[+0.333, +0.667]$ excludes zero. Of the two headline results, the misconception one is the one that holds up.

What it supports is narrower than "the graph teaches better". `full` differs from `rag_mc` in several ways at once—ordering block, forbidden list, layout rules, and a page of a particular kind. The defensible reading is that a contract specifying where corrective content must appear gets it to appear reliably, whereas the same content as flat prose does not; whether the active ingredient is the dedicated page, the surrounding structural demands, or the mention of a required slot is not identified by this design (§7.2).

(3) Ordering: where the representation runs out. `full` violates order at 0.020 $[0.000, 0.050]$, below the placebo (0.096) and `cot` (0.089), but `zero_shot`, told nothing about order, sits at 0.037 $[0.000, 0.096]$: a difference of -0.017 $[-0.086, +0.037]$, too wide to exclude a small effect in either direction. The ablation does not narrow it, because the arm built to remove the order does not remove it: the required-concept block is emitted in topological order—the same order the deleted block restates—so `no_order` still tells the model what comes first, and nine runs of a duplicate cannot price the original. This is the dimension §1.5 marked in advance, and the result matches the prediction: ordering is neither refuted nor confirmed—it is the dimension on which this representation stops being able to separate the requirement from the content that satisfies it, and we classify it as a boundary rather than price it as an effect.

(4) Out-of-graph: `rag` and `rag_mc` are worse than zero-shot. Both reach 0.111 and the placebo 0.167, whereas `zero_shot` sits at 0.019 and `full` at 0. Telling the model "this session covers these points" leads it to bring in adjacent advanced material—mentioning priority queues in a session on queues—because the prompt demarcates a scope without demarcating a boundary. `full` reaches zero only through the forbidden-term list, an explicit negative example, as the ablation confirms.

(5) Attribution: each knowledge component buys the metric it governs. Table 5 deletes one block at a time and re-measures every metric, so the ablation reads as an attribution rather than as a second set of condition means. The misconception slot is worth 0.889 of misconception coverage on its own, and its removal costs 0.037 of coverage as well, because corrective content that names the concept and its failure mode organises the key points by the way. The forbidden-term list is worth 0.083 of out-of-graph control and moves nothing else—a clean single-responsibility attribution. The ordering block moves its own metric and no other, which is the exception already discussed. Read literally, its row says that deleting the block improves order (0.026 against 0.000), and that is the reading to refuse: the row does not price the ordering constraint, because the deleted line duplicates an order the prompt still carries, so the manipulation removes nothing—and the -0.026 is not the block being harmful but two of the nine runs violating order, both on U11. The positive gains in this paper are attributable to named components rather than to the pipeline as a whole: each block moves the metric it governs and leaves the others at their floor or ceiling, with one spillover—the slot also carries 0.037 of coverage—and the one arm that moves nothing is the arm whose manipulation leaves its target in place.

(6) The code dimension: checked, not ranked. Every one of the 54 runs carries code—`full` on 48% of its pages against 27–33% for the baselines—and no headline metric reads it, so we report a secondary check rather than a score. All 117 blocks parse with Python's `ast` in every condition; a behavioural probe that strips top-level demo statements and interrogates up to four behaviours (FIFO order, the empty-dequeue signal, the full signal, survival of repeated enqueue) admits 37 blocks, of which 36 pass, the single failure being a genuine scaffolding defect—a program-style `enqueue(q, x)` referencing a capacity constant its own slide never defines. Parsing cleanly everywhere is consistent with teachers rating content accuracy at or above 4.0 in every condition (Table 9), and it means a runnability metric must test behaviour to

Table 5: Attribution by ablation: each block is deleted alone and all four metrics re-measured in the same batch ($n = 9$ per arm). Δ is the arm minus `full`; a block's contribution is the movement in the metric it governs. Secondary movement above 0.03 is shown as a side effect. The ordering row is a null manipulation rather than a measurement of the constraint (§1.5), and its -0.026 must not be read as "deleting the block helps": the two order-violating runs of this batch both sit in the `full` arm (on U11), which is also why every other arm reads 0.000. The row prices nothing; it shows only that the interface cannot withhold the order. Mean (standard deviation) over resamplings.

Block removed	Metric	`full`	Without	Δ
Misconception slot	Misconception ↑	1.000 (0.000)	0.111 (0.333)	−0.889
Misconception slot	Coverage ↑ (side)	0.981 (0.056)	0.944 (0.083)	−0.037
Forbidden list	Out-of-graph ↓	0.000 (0.000)	0.083 (0.125)	+0.083
Ordering block†	Order viol. ↓	0.026 (0.063)	0.000 (0.000)	−0.026

discriminate at all. The probe is a bound rather than a metric: the harness is ours, so the evaluator supplies part of the answer it then checks, and 8 of 9 against 28 of 28 separates nothing. We publish it because a criterion for a programming course that cannot see a broken `enqueue` is the honest limit of this evaluation, not because it ranks the arms (§7.6; Table S3).

`full` consumes 1064 input and 4079 output tokens (2708 of them reasoning) at 20.8 s end to end, against 347, 2330, 984 and 12.4 s for `zero_shot`: about 1.75 times the output, the surplus being the contract block (roughly 500 input tokens) and fuller speaker notes—the page count is fixed by `slide_budget` and met in every run. The placebo is the informative contrast on input: 1252 tokens, more than `full`, with only 111 reasoning tokens. Per deck the cost stays in the cents range, negligible against preparation time, but for course-wide batch generation the 1.75× gap is worth engineering away, for example by caching contract blocks per session.

5.5. Crossing the factors: the slot needs the text and *the contract*

The conditions in Table 4 vary one thing at a time, which prices each block but cannot say whether a block's effect depends on another's. The misconception result is the case where that question has an obvious answer available: the control that receives the same misconception text without the contract (`rag_mc`) already reaches 0.556, so the slot's 1.000 could be the contract delivering a text the model cannot otherwise use, or the text alone,

Table 6: The 2×2 behind the misconception result: misconception coverage, mean (standard deviation) over nine runs (three sessions × three resamplings) per cell. "Structure" is the contract—concept list, ordering, forbidden terms—plus the instruction to dedicate a page to the misconception; "text" is the misconception statement and its correction. Generated by `paper/factorial.py`.

	misconception text	
	present	absent
contract structure	1.000 (0.000)	0.111 (0.333)
no contract structure	0.556 (0.527)	0.222 (0.441)

or both. Crossing the two factors separates them, and the four cells needed are four conditions we had already run: `full` and `no_misconception` on the structure side, `rag_mc` and `rag` on the text side (Table 6).

The cell means give a main effect of +0.167 for structure and +0.611 for the text, and an interaction of +0.556. The interaction is easiest to read as two simple effects of the same text: with the contract in place the misconception material is worth +0.889 (1.000 against 0.111), and without it +0.334 (0.556 against 0.222). The contract on its own is worth −0.111—one run out of nine, and not a difference to build on. Were the two additive, the joint cell would sit at 0.445; it sits at 1.000. The contract is not delivering the text; it is making the text land. Neither factor alone is close to the joint result, so the slot's reliability cannot be attributed to the information it carries—which is the objection `rag_mc` was built to raise, answered by crossing rather than by argument.

The inference is deliberately weak. With three sessions the unit of analysis is the session, and the per-session interaction is positive in all three (+0.333, +0.667, +0.667), but an exact sign-flip test over three units can return no smaller two-sided p than $2/8 = 0.25$, and it returns exactly that: the pattern is consistent and the design cannot certify it. Two further limits are worth stating. The structured arm differs from the unstructured one in the layout instruction as well as the text, so the factor is the *requirement*—text plus "dedicate a page to it"—not the text alone. And the four cells are four conditions that were run for other purposes, not a crossing randomised inside one batch: `no_misconception` comes from the ablation batch while the other three come from the main one, which is why Table 5 reports that batch's `full` arm beside it. The two batches agree to three decimals on this metric

(1.000 ± 0.000 in both), so a between-batch difference is not what produces the interaction—but agreement on one arm is not a common batch. Running the four cells together is the clean version of this design, and it is not what we did; what the table supports is a mechanism, not a significance claim.

5.6. A second backbone replicates the contrast

Every number above comes from one vendor's model, which leaves the obvious objection: a contract might be compensating for one model's prompt tuning rather than supplying knowledge. We therefore re-ran the four claim-bearing conditions on Kimi K3—same prompts, sessions and validator, six runs each, five for zero-shot, which is reported as missing rather than refilled (Table S4 in the supplementary material). That vendor pins sampling, so the replication runs at temperature 1.0 against 0.3; within-model contrasts are unaffected, cross-model levels are not compared.

The sign replicates on both metrics that carry the paper. Misconception coverage falls from 1.000 under `full` to 0.667 under `rag_mc` (-0.333, against -0.444 on DeepSeek) and to 0.333 for the length-matched placebo; coverage falls from 1.000 to 0.734 (-0.266, against -0.283). More telling than either mean is the variance: `full` returns 1.000 in every run under both models, while `rag_mc`'s standard deviation is 0.516 on Kimi against 0.527 on DeepSeek. What replicates is not an effect size but a property: a contract that names the slot fills it every time, and the same text without a slot fills it two times in three.

Two things do not replicate, and they bound the claim. The gain is smaller on the second model (0.333 against 0.444), which is what a stronger model should produce if the contract compensates for a weakness rather than teaching something the model cannot do; and on U11 every condition scored 1.000 on Kimi, zero-shot included, so the session that carries the effect under one model carries none under the other. Six runs also leave the pass-rate gap (67% against 17%) without a usable interval. The replication rules out a single vendor's artefact and supports nothing about model families; one more model would not change that.

5.7. A second course: the direction replicates, the reliability tracks the backbone

Every result so far comes from one course, so the natural objection is that the contract does nothing a competent teacher's prompt would not do

Table 7: Second course (*C++ Programming*, arrays chapter), three conditions × two sessions × two resamplings ($n = 4$ per condition), a different backbone and endpoint from Table 4. Mean (standard deviation). Generated by `paper/tables.py --course-b outputs/bench_cpp`.

Condition	Coverage ↑	Out-of-graph ↓	Order viol. ↓	Misconception ↑
full (ours)	0.800 (0.163)	0.000 (0.000)	0.042 (0.083)	0.625 (0.479)
rag_mc	0.600 (0.231)	0.000 (0.000)	0.017 (0.033)	0.375 (0.479)
zero_shot	0.350 (0.191)	0.125 (0.250)	0.042 (0.083)	0.000 (0.000)

on a different syllabus. We therefore replicated the two claim-bearing conditions on a second course: the arrays chapter of *C++ Programming*, taught in the same department to a different cohort, with a graph built from that course's own lesson plan—the same object type, different knowledge, different language. Two sessions, three conditions (`full`, `rag_mc`, `zero_shot`) and two resamplings: twelve runs. Two things differ from the main study, and both are stated rather than smoothed over: the backbone is a different model on a different endpoint (`agnes-2.5-flash`, temperature 0.3), chosen because the account's primary API key had exhausted its quota when the runs were issued; and the course's misconceptions come from the lesson plan's difficulty list and reflection section rather than from exam error statistics—instructor-documented difficulties, not measured frequencies. The replication therefore tests course and backbone jointly, and its magnitudes must not be read against Table 4.

The direction survives. Coverage is 0.800 (0.163) against 0.350 (0.191) for zero-shot and 0.600 (0.231) for the text-matched control—gaps of +0.450 and +0.200, larger than the +0.283 of the first course because the weaker backbone starts lower. Misconception handling is 0.625 (0.479) against 0.000 (0.000) for zero-shot and 0.375 (0.479) for the control given the same misconception text. Out-of-graph behaviour reproduces the first course's pattern too: zero-shot imports material from later chapters (0.125), the contract does not (0.000). Both claim-bearing contrasts keep their sign on a different course, a different language and a different backbone.

The second moment does not. In the first course the slot produced 1.000± 0.000; here it produces 0.625 ± 0.479—two runs of four fill the slot and two do not, and the matched control has the same dispersion. The contract still moves the mean, and still beats the control that carries the same text,

but it does not make the outcome certain. Two explanations were available: the backbone is weaker on the task (its zero-shot coverage is 0.35 against 0.64, and it is the only condition here that imports out-of-graph content), or the misconceptions of this course are documented difficulties rather than measured ones, so the prompt states what students get wrong without stating how often.

The mirror cell separates them, and not in the direction the first explanation predicts. On this course under DeepSeek the slot reaches 0.875 ± 0.250 in four runs: better than under `agnes-2.5-flash`, and still not certain, where the same endpoint returns 1.000 ± 0.000 in nine runs out of nine on the first course. The residual variance travels with the syllabus, not only with the model: a compliant backbone fills the slot on the course whose misconception counts the graph records, and fills it in most runs rather than all on the course where those counts are assumed. That is a narrower condition than "a good model suffices", and a more useful one, because a course's misconception inventory is something an instructor can fix.

Ordering is the third check, because that is where the scope claim makes a prediction. Violations are 0.042 (0.083) for `full` and `zero_shot`, with 0.017 for `rag_mc`: no contrast, on a course whose textbook order is canonical as well—the second independent reading of the same boundary (§1.5). What the second course buys is therefore narrower and more useful than "it generalises". The *direction* of both claim-bearing contrasts is course-independent; the *reliability* claim is conditional twice over—on a backbone able to execute a structural instruction, and on a course whose misconceptions are counted rather than assumed. Neither condition is visible inside one course, and both are the kind of statement a reader can check against their own model and their own syllabus.

Which of the two changed. The second course moves syllabus, language and endpoint at once, so the failure above has two candidate explanations and its own design cannot choose between them. Holding the first course unchanged in every respect, we re-ran the same three conditions on `agnes-2.5-flash`: three sessions, two resamplings, 18 runs, Table 8 row 3. The slot's advantage disappears there too, and in the same way: 0.833 ± 0.408 against 0.833 ± 0.408 for the control that receives the same misconception text, a difference of 0.000 where the same course produced +0.444 under DeepSeek and +0.333 under Kimi. Only the endpoint changed, so the precondition tracks the model rather than the course; the second course's failure was most plausibly this behaviour, encountered on a different syllabus. That

sentence is the strongest inference in the paper and should not be read as more than it is: Table 8 is five cells with two resamplings each—four runs per condition—compared descriptively; nothing here is an interaction test, and a three-cluster interval on any of those differences would include zero.

Why not the full crossing. A course × backbone 2×2 would need a third course graph authored from another syllabus before the first API call; the compute for such a grid costs less than a dollar, so the binding constraint is authoring, not inference. That is why the design grows one cell at a time, why the fifth cell answers the attribution question in sign rather than in size, and why estimating the reliability condition—rather than ordering it—is the first item of future work we would fund.

The fifth cell is the one that was missing, and it points the same way. Holding the endpoint fixed and changing the course, the contrast does not disappear: on the second course under DeepSeek it is +0.500 for misconceptions and +0.200 for coverage, against +0.444 and +0.283 on the course it was found on. Read the cells by endpoint and the misconception contrast is +0.444 and +0.500 under DeepSeek, +0.333 under Kimi, +0.000 and +0.250 under `agnes-2.5-flash`; read them by course and the two C++ cells straddle the Python ones. The endpoint orders the cells and the course does not, which is the direction the precondition predicts—but with four runs per condition this is an ordering, not an estimate, and it does not say which model property does the work. What the comparison establishes is that the contrast is weakest exactly where the endpoint resists a structural instruction, on either syllabus, and that it removes the syllabus as the explanation without naming its replacement.

Two details keep this from collapsing into "the model decides". The forbidden-term requirement still binds on the same runs—out-of-graph content falls from 0.097 under the text-matched control to 0.000, exactly the pattern of the first course under DeepSeek—so what this backbone does not do is fill a required *page*; it is not indifferent to structure in general. And the coverage margin over the text-matched control falls to +0.052 on this backbone while the margin over zero-shot stays at +0.401: on a model that already reads the supplied text carefully, a contract that supplies the same text buys less. Neither observation is a mechanism—two runs per cell make them patterns, and we report them as patterns.

Table 8: The same two paired contrasts on every (backbone, course) cell we have. Rows 1–3 hold course, graph, prompts, validator and metric code fixed and vary only the endpoint; rows 4 and 5 hold the endpoint fixed and vary the course (row 4 against row 1, row 5 against row 3). Δ is `full` minus `rag_mc`; the last column is the distribution the reliability claim is about. Two resamplings per cell, four runs per condition. Generated by `paper/tables.py --sweep`.

Backbone	Course	Δ coverage	Δ misconception	`full`'s misconception
DeepSeek V4	Python	+0.283	+0.444	1.000 (0.000)
Kimi K3	Python	+0.266	+0.333	1.000 (0.000)
agnes-2.5-flash	Python	+0.052	+0.000	0.833 (0.408)
DeepSeek V4	C++	+0.200	+0.500	0.875 (0.250)
agnes-2.5-flash	C++	+0.200	+0.250	0.625 (0.479)

5.8. Teacher blind review

The six metrics measure conformance to the contract, not teaching quality (§7). To check that the two agree, ten decks—three from each of `full`, `rag_mc` and `rag`, one from `placebo`—were scored independently by three teachers who did not know which condition had produced a deck, each deck rated on five five-point items and on an overall score, defined as the unweighted mean of the five. Table 9 reports the result, and the raw ratings are released with the repository.

Protocol. Decks were exported to PDF with speaker notes removed and file names replaced by opaque identifiers, presented in one folder in an order fixed before scoring, with no condition label and no run identifier. The raters are three teachers of the same department who teach this course or its prerequisite. They scored in one sitting, independently, without discussing items or comparing decks, and were told only the course, the session and the item wording—not the research question, not the automatic metrics, and not that the decks came from more than one method. Items are anchored 1 (absent) to 5 (fully adequate) and reported verbatim in Table 9; the overall score is their unweighted mean, which `human_eval.py` verifies to 10^{-4}. No rater was paid and all three consented. We ran no training or calibration round, which is a real limitation: without one we cannot separate disagreement about the rubric from disagreement about the decks, and the pedagogical-logic ICC of 0.53 may reflect either.

Table 9: Teacher blind review on a five-point scale, mean (standard deviation) over ratings. Three decks per condition except `placebo` ($n = 1$ deck, descriptive only). ICC(2,3) is the agreement of the average of three raters. Content accuracy has no between-deck variance to model ($F(9, 18) = 1.00$), so its ICC is zero by construction rather than by disagreement.

Item	`full`	`rag_mc`	`rag`	`placebo`	ICC(2,3)
Syllabus coverage	4.78 (0.44)	3.56 (0.53)	3.33 (0.50)	3.67 (0.58)	0.80
Misconception correction	4.89 (0.33)	3.22 (1.48)	2.11 (0.60)	2.67 (0.58)	0.91
Pedagogical logic	4.56 (0.53)	3.78 (0.44)	3.67 (0.50)	3.67 (0.58)	0.53
Content accuracy	4.56 (0.53)	4.22 (0.44)	4.11 (0.33)	4.00 (0.00)	0.00
Classroom readiness	4.67 (0.50)	3.33 (0.50)	3.11 (0.33)	3.00 (0.00)	0.91
Overall	4.69 (0.28)	3.62 (0.47)	3.27 (0.24)	3.40 (0.20)	**0.92**
Decks	3	3	3	1	
Ratings	9	9	9	3	

Agreement is high where it matters. Averaging three raters gives an overall ICC(2,3) of 0.92 (95% CI $[0.69, 0.96]$, bootstrap over decks), strongest on the two items the manipulation targets—misconception correction (0.91) and classroom readiness (0.91)—with coverage next at 0.80. The two low values are informative: content accuracy has no between-deck variance to model (every condition averages at least 4.0, $F(9, 18) = 1.00$), so its ICC is zero by construction rather than by disagreement; pedagogical logic (0.53) is where raters diverge most, plausible since judging sequencing from slides requires knowing what the course already covered and the raters were not told.

The ordering is unambiguous: `full` averages 4.69 (0.28) against 3.62 (0.47) for `rag_mc`, 3.40 (0.20) for `placebo` and 3.27 (0.24) for `rag`, and every `full` deck (4.53–4.80) outscores every `rag_mc` deck (3.27–4.00). Complete separation of three against three gives a two-sided permutation $p = 0.10$, the smallest attainable at this sample size, so we read the human data as *consistent with* the automatic results rather than as confirmation at conventional significance. The agreement is more specific than a shared ranking: rescaling the misconception item to $[0, 1]$ gives 0.97, 0.56, 0.42 and 0.28 against automatic values of 1.000, 0.556, 0.333 and 0.222—identical rank order, gap at most 0.09. Content accuracy stays at or above 4.0 everywhere, echoing the automatic finding that the baselines are accurate but incomplete. Teachers localise the difference where the metrics do—not in whether the slides are

right, but in whether they teach what this class needs. Three limits apply: the raters are one cohort at one institution; three decks per condition permit description but not inference; and the `placebo` arm contributes one deck, so its column carries no weight.

5.9. Qualitative observations

Two cases anchor the numbers; the full readings are in the supplementary material. In the hardest session (U12), all three `full` runs reach coverage 1.000 and each produces a genuine comparison slide, where the five baselines on the same session average 0.333. And `full` fails too: one U10 run never defines "queue" beyond one sentence on the title slide—a convention-dependent miss that §5.3 prices and anyone who admits cover definitions may read as a pass—while a second U10 run contains two prerequisite inversions, which no convention excuses: the topological order was in the prompt and the model violated it anyway, which is why we insist on post-hoc validation rather than assuming that saying it implies doing it.

6. Discussion

6.1. Where the graph helps and where it does not

Rearranging the six metrics by "does the graph make an irreplaceable contribution" sorts them into four groups. *Irreplaceable*: misconception handling (1.000 ± 0.000 against 0.556 ± 0.527 with the same text) and coverage completeness (0.981 against 0.61–0.70, the length-matched placebo excluding prompt length). *Partial*: scope control, resting on the forbidden-term list as an explicit negative example (ablation $0.000 \rightarrow 0.083$). *Beyond this representation*: prerequisite order (0.020 against 0.037 for zero-shot, the ablation running the other way, for the reason given in §1.5). *Orthogonal*: editability, 1.000 everywhere and determined by the renderer.

The first two groups are a result about knowledge; the third is a result about representation. The graph supplies not disciplinary knowledge but the layer of institutional convention around it. A strong model already handles textbook content and textbook order; which concepts count as "must be finished in this session," where this cohort errs, and what belongs to the next session appear in no pretraining corpus, and once supplied the model uses them stably. Order is the exception that tests the rule: the only order worth constraining is institutional, and that is precisely the kind the interface cannot deliver.

The pattern is that actionability follows the form of the requirement: a dimension enters the criterion when it can be written as a condition on the *structure* of the artefact—an element that must be present, one that must be absent, a page of a given kind—and stays outside when the only honest statement is a relation between elements the interface has already flattened into a list. Narrowness, on this reading, is a fact about which curriculum dimensions have a structural form, not about how much of the course we sampled. *Cohort-specific misconceptions* carry the strongest evidence: given the text, `rag_mc` writes a correction slide in about half of nine generations and folds it into a bullet otherwise, after which it disappears; given the structure, `full` converts it into a layout-level requirement and is stable. Misconception knowledge must not only be stated, it must be stated *into position*. *Skill landing points* come next: baselines omit skills such as implementing enqueue and dequeue on a list—not because the model cannot write code, but because it does not know this session requires *writing* rather than *understanding*; the 0.98 against 0.67 coverage gap comes largely from here. *Institution-specific order* is out of reach for this representation: the module's order matches the standard textbook, so a silent baseline is not disadvantaged, and the contract cannot withhold the order it states (§1.5). A syllabus that *deliberately departs* from textbook order would remove the first confound; the second needs an interface that can present concepts without sequencing them—a shuffled list, or a contract that names the successor relation. That is the follow-up we consider most worth doing, and it is a change to the representation, not to the sample.

6.2. What the graph changes: reliability, not average quality

A system that produces a good deck two times in three is not a cheaper teacher; it is an unusable one, because the class arrives on schedule and a failure is absorbed by the person who has to prepare the session again. What separates the conditions is not how much they say—`rag_mc` receives the same misconception sentence, in prose—but that under the contract the sentence becomes a *requirement with a place*, so an omission is a defect to the same program that asked for the content.

The informative difference is in the second moment. `rag_mc` averages 0.556 ± 0.527; `full` averages 1.000 ± 0.000. A teaching workflow cannot absorb a coin flip, so evaluation in this area should report variance, not means alone, which the comparable literature we surveyed frequently does not. Table 5 also says which part of the contract buys that reliability: deleting the

misconception slot alone returns the metric to 0.111, so the stability is a property of a component a reader can inspect and re-derive, not of the prompt as an undifferentiated whole.

The second course puts a condition on the sentence: there the slot moved the mean but not the dispersion (0.625 ± 0.479, §5.7), and re-running the first course under that backbone reproduces the disappearance with nothing but the endpoint changed. The fourth cell of Table 8 splits the two readings: the contrast travels with the syllabus, the residual variance does not travel with it alone. Reliability is a property of the pair—the representation *and* the executor—and reporting it without naming the pair would repeat the mistake this section is about.

The same logic makes the negative results load-bearing: no gain from ordering, no discrimination from editability, and a 33% non-pass rate for `full` itself are credibility signals, because the metrics do not systematically favour the constraint-based condition (seven defects were fixed to that end, §7.1). The ordering null is the clearest instance—it marks where the representation ends, which no table of wins could say.

7. Threats to validity

7.1. Text-level metrics systematically favour constraint-based methods

This is the most important methodological caution in the paper. During the study we discovered and corrected seven measurement defects; each made the `full` condition win without cause, not one raised an error, and none was found by reflection—all came from printing model outputs slide by slide and reading them. The common structure across all seven is that the representation the constrained method receives is exactly the representation the metric uses: verbatim coverage matching, order scored from self-reported IDs, preview slides counted as body, a forward reference treated as teaching, cross-slide subtraction, over-broad misconception anchors, and an out-of-graph test that punished a different ID namespace. Each is now pinned by a regression test in `tests/test_measurement_defects.py` (the suite stands at 44 tests), and Supplementary Section S9 lists symptom, consequence and fix for all seven.

Two readings matter in the main text. The 0.833 for `full`/U10/r3 in Table 4 is *not* a surviving instance of the fifth defect but a genuine omission of the definition of "queue" (§5.9). And the sixth was not a bug in metric code but an error in the graph data: one mis-attached "related" edge made

any misconception-layout slide mentioning empty-queue detection count as addressing a different misconception, the ablation still scored full marks, and the conclusion about misconception injection nearly became false. Graph errors silently contaminate metrics—a risk specific to graph-driven methods, where the knowledge base is both the object of study and part of the measuring instrument; we therefore recommend reporting how the metrics behave *on the baselines* and doing at least one manual pass with outputs printed slide by slide.

7.2. Sample and external validity

Three sessions in one course, two in a second course in another language, three endpoints on the first course and one of them on the second, two to three resamplings per cell: this supports descriptive reporting and does not license significance claims. The second course contributes four runs per condition—enough to see that a distribution became two-of-four, not enough to estimate how often it does so: a sign, not a rate. Eight gaps follow.

Power, stated as arithmetic rather than assumed. A two-sided sign-flip test over sessions cannot return a p below $2(1/2)^k$ with k sessions: three sessions give a floor of 0.25, and the floor falls below 0.05 only at six. With nine runs nested in three sessions the design can therefore detect only a difference large enough to hold in every session; a smaller one is not "not significant" but unmeasurable, and reporting a p value from it would restate the arithmetic above rather than add evidence. This is why the intervals in §5.4 are descriptive and why a revision that added asterisks would weaken the paper rather than strengthen it.

One factor at a time. Each pair varies one thing while holding another fixed. Only the misconception result is crossed (§5.5), and there the two factors turn out not to be additive at all: the interaction is +0.556, so the marginal reading—both controls individually null, effects additive—is wrong for that component. That crossing is also post hoc, assembled from conditions run for other reasons and split across two batches, so it is a mechanism check rather than a pre-registered contrast. Structure is likewise not one thing: the structured arm changes the concept list, the ordering, the forbidden terms and the layout instruction together, so a "structure" main effect summarises that bundle rather than any one part.

Two models, not a model family. All runs come from hosted endpoints at one point in time. §5.6 re-runs the claim-bearing conditions on a second vendor's model and the contrast survives, but two draws do not characterise

a space: ordering was already handled well by both models, and we have no evidence about weaker open-weight models, where the graph might help on ordering too, or stronger ones, where the misconception gap already narrowed from 0.444 to 0.333 and could close. The study is reproducible in procedure and not in output: a hosted endpoint may retire a model without notice, and Kimi K3 already refuses the temperature the main table uses.

Course and backbone, separated and only partly crossed. The second course changes syllabus, language and endpoint together, which on its own would leave the failure of variance replication unattributable. §5.7 closes that hole from one side: re-running the first course unchanged under the second course's backbone reproduces the failure, so the condition sits with the model. The reverse cell in Table 8 splits the two readings rather than repeating one: the second course under DeepSeek keeps the contrast (+0.500 for misconceptions, +0.200 for coverage), so the syllabus did not produce the contrast—and it still does not reach zero dispersion (0.875 ± 0.250 against 1.000 ± 0.000), so the syllabus carries part of the residual variance. Four runs per condition answer the interaction in sign, not in size.

Two courses, two modules. Both modules are unusually misconception-rich (empty/full, false overflow and BFS semantics in the first; 1-based indexing and the null terminator in the second), close to a best case. A course with a thin misconception inventory should show a smaller effect, and we claim nothing about its size.

One graph, one author. The graph was authored by the instructor who teaches the course; its edges were not independently annotated, so we cannot report inter-annotator agreement on the graph itself. Defect 6 is the concrete consequence—one mis-attached edge nearly inverted a headline result.

No learning outcome. Nothing here measures whether students learn more from a constrained deck. Every metric, including the teacher review, scores the artefact; the chain from artefact quality to learning gain is untested, and it is the chain a teacher cares about.

Under-powered ablations, one of them null. Nine runs per arm separate a metric that moves from 0.000 to 0.083 and cannot resolve one that moves by 0.026; and the ordering arm removes less than its name claims, since the concept list it leaves behind is already topological (§5.4). We treat this as a limit of the interface disclosed in §1.5 rather than a gap more runs would close: more runs would estimate the price of a duplicate more precisely, not the price of the constraint.

7.3. What the human review can and cannot support

The blind review of §5.8 narrows this gap without closing it. Ten decks, three raters, one institution and a single `placebo` deck support a consistency check, not a validation study; the human–automatic agreement is measured at condition level on four points, enough to say the two orderings coincide and not enough to estimate a correlation. Nor do the items exhaust teaching quality: clarity of board work, appropriateness of examples for this cohort, and the fit between slides and the teacher's own voice remain outside both the metrics and the rubric. What the review establishes is that teachers order the conditions as the metrics do and locate the difference on the same axis—misconception handling and coverage—while rating every condition as factually accurate. A larger review with raters from other institutions is the obvious next step and the place where our evidence is thinnest.

7.4. A repair loop, and where it stops

One third of the `full` runs carry an error-level finding, so we tested whether the validator's findings can be fed back (policy and per-run outcomes in Supplementary Section S11). On the main grid one regeneration round repaired all three failures (6/9 → 9/9; mean coverage 0.981 → 1.000), and the repairs *added* content rather than removing it: a missing concept got the page it lacked, an inverted pair was fixed by splitting a merged page. On `agnes-2.5-flash` the same policy moved 4/6 to 5/6 and left the ordering failure unrepaired after three attempts (0.048 → 0.048 → 0.381)—three attempts produced three decks with no convergence, so a repair budget has to be set by cost, not by the expectation of convergence. The loop closes every failure on the backbone that fills a required slot and one of two on the backbone that does not, which is the paper's precondition seen a third time; 9/9 is therefore reported as a bounded, costed repair on nine decks, not as the pass rate of the method.

7.5. Evaluation independence

The first author wrote the graph, the method, the metrics and the rubric; the co-authors contributed validation, formal checks and revision, but none wrote the metric code—the standard shape of a systems paper and its standard weakness. Three countermeasures were taken and none substitutes for an independent coder. Every metric is computed by released code against a released contract, and the seven defects were found by printing model output and reading it rather than by inspecting the code. The rubric was applied

identically to all conditions with one shared vocabulary, baselines were never told which fields the metrics read, and the `placebo` arm exists to test whether the metric measures prompt length and fails to explain the gain. The blind review puts the ranking question to three raters who did not know the conditions, and they reproduce it. The rubric's sensitivity is reported rather than defended: two of its three decisions were re-run under the alternative convention (§5.3), the conclusion survives both, and the reported advantage sits on the conservative side. What is missing is the obvious thing—a second coder who did not write the metric, coding a subsample against the same rubric with disagreement reported. Given three decisions that materially move the numbers, that check would be worth more than a larger sample scored by the same author. To make it cost a coder-day rather than a new study, the released artefact carries the packet: a stratified subsample of the main grid under opaque identifiers, the rubric with its anchors, and the script that reports weighted kappa, ICC and Krippendorff's alpha once a second coder returns the sheet (`paper/independent_coding/`).

7.6. Code is parsed, and only partly executed

For a programming course this is the weakest joint, and it is a limit of the criterion rather than of the pipeline: the acceptance test is textual, so a deck can score coverage 1.000 while its `enqueue` has an off-by-one and nothing in Table 4 moves. The runnability check of §5.4(6) is the most we claim—one syntactic pass over all 117 blocks and one bounded behavioural probe whose denominator is too small and too condition-dependent to rank arms. It establishes that the code dimension is not silently broken, not that it was compared; a comparison would need a per-session harness and test set, released.

7.7. The constructedness of the misconception result

`misconception_coverage` is structurally unfavourable to baselines, since they were never told about the misconceptions. Our response is the `rag_mc` control, which supplies the same text in flat form. If `full` still wins, the gain comes from structure rather than information. Even so, we must concede that the metric measures *whether* the course-required correction was made, not *whether it is correct*.

8. Conclusion and future work

We reframe slide generation as a course-specification satisfaction problem and make the criterion, not the generator, the contribution: one graph-derived contract serves as generation basis and acceptance ground truth, so what is demanded and what is checked are the same object and cannot drift, and alignment is measured per artefact instead of asserted per course design.

Empirically the gain is a gain in *reliability*—a misconception slot that is always filled rather than usually filled—and the knowledge the graph makes irreplaceable is *the skill landing points in the syllabus* and *the cohort-specific misconceptions*, neither of which exists in any pretraining corpus. Each gain is priced against a named block rather than against the pipeline as a whole (Table 5), which is what lets a reader inspect the claim instead of taking it on trust. The same study locates the edge of that claim: order is expressible in the graph but cannot be withheld by the interface, so it is reported as the boundary of the representation rather than as a result about ordering.

Future work proceeds along six lines: (i) an interface that can present concepts without sequencing them, so that ordering becomes testable at all; (ii) a course $\times$ backbone grid with a third course graph, so that the reliability condition is estimated rather than ordered (§5.7); (iii) extend to all sessions of the course and widen the blind review to raters from other institutions, courses and vocational disciplines, with a second coder who did not write the metrics; (iv) build graphs semi-automatically from textbook PDFs to reduce authoring cost; (v) add a render-and-review loop (screenshot, detect overflow and overlap, repair automatically); and (vi) release a Chinese vocational slide-generation benchmark so that comparable work can be compared on common ground.

CRediT authorship contribution statement

Suxing Liu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization.

Byungwon Min: Conceptualization, Supervision, Project administration, Writing – review and editing.

Zhijian Zheng: Validation, Formal analysis, Writing – review and editing.

Jiahao Wang: Investigation, Data curation, Visualization, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

Code, both curriculum graphs, prompts, experimental protocol and all raw model outputs are archived in the public repository `https://github.com/HAHA1122344/kg-slidegen`, at the tag `v1.4`, which is frozen for review and carries the full development history behind it. Each experimental cell retains three artefacts: the rendered deck, the raw model output, and an itemised validation report. Every table in this paper and in the supplementary material is regenerated from that tree by the scripts in `paper/` without calling any model, and `tests/` reproduces the graph's own validation results (Table S2). The same tree is additionally packaged as a single archive, and its `sha256` is committed in `paper/deposit/SHA256SUMS`: packaging the tree twice yields the same bytes, so a reader can confirm that what they hold is the reviewed artefact rather than a later rebuild.

Supplementary material

The supplementary material (Sections S1–S12) holds the ablation results (S1), the mutation test of the graph's structural checks (S2), the full behavioural probe of the generated code (S3), the cross-model replication (S4), the concept-level paired analysis with its leave-one-concept-out check (S5), verbatim qualitative readings of the two decks discussed in the qualitative overview (S6), the full prompt for one session (S7), one raw model response reproduced in full (S8), the seven measurement defects with symptom, consequence and fix (S9), the recomputation of the two coverage conventions (S10), the repair loop with its policy and per-run outcomes (S11), and the reproducibility record with its commands and hashes (S12)—together with the scripts that regenerate each of them from the archive.

References

[1] H. Liang, Q. Lin, Z. Han, X. Ma, Z. H. Wong, M. Qiang, L. Sun, W. Zhang, K12-KGraph: A curriculum-aligned knowledge graph for benchmarking and training educational LLMs, in: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2026, arXiv:2605.09635, v4 of 3 Oct 2026; the graph and K12-Train grew between v1 and v4. doi:10.48550/arXiv.2605.09635.

[2] X. Dai, Y. Hua, T. Wu, Y. Sheng, Q. Ji, G. Qi, Large language models can better understand knowledge graphs than we thought, Knowledge-Based Systems 312 (2025) 113060. doi:10.1016/j.knosys.2025.113060.

[3] W. Huang, G. Zhou, M. Lapata, P. Vougiouklis, S. Montella, J. Z. Pan, Prompting large language models with knowledge graphs for question answering involving long-tail facts, Knowledge-Based Systems 324 (2025) 113648. doi:10.1016/j.knosys.2025.113648.

[4] J. Ma, Z. Zhao, Z. Xie, Y. Zhang, G. Zhou, LE-DLCM: Decoupled learner and course modeling with large language models for enhanced course recommendation, Knowledge-Based Systems 334 (2026) 115135. doi:10.1016/j.knosys.2025.115135.

[5] J. Roschelle, E. A. McLaughlin, K. R. Koedinger, Beyond benchmarks: Responsible AI in education needs learning sciences, Communications of the ACM 68 (12) (2025) 37–39. doi:10.1145/3747174.

[6] T.-J. Fu, W. Y. Wang, D. McDuff, Y. Song, DOC2PPT: Automatic presentation slides generation from scientific documents, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022. doi:10.1609/aaai.v36i1.19943.

[7] I. Mondal, S. Shwetha, A. Natarajan, A. Garimella, S. Bandyopadhyay, J. Boyd-Graber, Presentations by the humans and for the humans: Harnessing LLMs for generating persona-aware slides from documents, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), St. Julian's, Malta, 2024, pp. 2664–2684. doi:10.18653/v1/2024.eacl-long.163.

[8] J. Ge, Z. Z. Wang, X. Zhou, Y.-H. Peng, S. Subramanian, Q. Tan, M. Sap, A. Suhr, D. Fried, G. Neubig, T. Darrell, AutoPresent: Designing structured visuals from scratch, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025, pp. 2902–2911. doi:10.1109/CVPR52734.2025.00276.

[9] H. Zheng, X. Guan, H. Kong, W. Zhang, J. Zheng, W. Zhou, H. Lin, Y. Lu, X. Han, L. Sun, PPTAgent: Generating and evaluating presentations beyond text-to-slides, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Suzhou, China, 2025, pp. 14402–14418. doi:10.18653/v1/2025.emnlp-main.728.

[10] Y. Guo, Z. Zhang, Y. Liang, D. Zhao, N. Duan, PPTC benchmark: Evaluating large language models for PowerPoint task completion, in: Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, 2024, pp. 8682–8701. doi:10.18653/v1/2024.findings-acl.514.

[11] X.-S. Chen, J. Zhu, P.-l. Li, H. Wang, S. Yang, M. Guo, PresentBench: A fine-grained rubric-based benchmark for slide generation, 238 instances, 54.1 checklist items per instance (2026). arXiv:2603.07244, doi:10.48550/arXiv.2603.07244.

[12] D. Jang, M. L. Heisler, L. Xing, Y. Li, E. Wang, Y. Xiong, Y. Zhang, Z. Fan, DeckBench: Benchmarking multi-agent frameworks for academic slide generation and editing (2026). arXiv:2602.13318, doi:10.48550/arXiv.2602.13318.

[13] Z. Huang, X. Liu, T. Hu, K. Zhang, Y. Liu, PPTBench: Towards holistic evaluation of large language models for PowerPoint layout and design understanding (2025). arXiv:2512.02624, doi:10.48550/arXiv.2512.02624.

[14] K. Kumar, C. R. Chowdary, SlideSpawn: An automatic slides generation system for research publications (2024). arXiv:2411.17719, doi:10.48550/arXiv.2411.17719.

[15] K. Jung, H. Cho, J. Yun, S. Yang, J. Jang, J. Choo, Talk to your slides: High-efficiency slide editing via language-driven structured data

manipulation, in: Findings of the Association for Computational Linguistics: ACL 2026, San Diego, California, 2026, pp. 3370–3399. doi:10.18653/v1/2026.findings-acl.166.

[16] T. Lu, Z. Wang, Y. Zhou, Y. Guan, Z. Bai, J. Du, SciMKG: A multimodal knowledge graph for science education with text, image, video and audio, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 15466–15474. doi:10.1609/aaai.v40i18.38574.

[17] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization (2024). arXiv:2404.16130, doi:10.48550/arXiv.2404.16130.

[18] Y. Huang, J. X. Huang, A survey on retrieval-augmented text generation for large language models, ACM Computing Surveys 58 (12) (2026) 1–38. doi:10.1145/3805774.

[19] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations, 2024. doi:10.48550/arXiv.2310.11511.

[20] S. Santschi, Think tasks, not tools: why AI policy in higher education needs constructive alignment, Teaching in Higher Education, advance online publication (2026). doi:10.1080/13562517.2026.2671391.

[21] B. Divjak, B. Svetec, D. Horvat, N. Kadoić, Assessment validity and learning analytics as prerequisites for ensuring student-centred learning design, British Journal of Educational Technology 54 (1) (2022) 313–334. doi:10.1111/bjet.13290.

[22] C. Wekerle, M. Sailer, I. Kollar, T. Fütterer, The effects of ICAP learning activities on learning outcomes, Learning and Instruction 107 (2026) 102482. doi:10.1016/j.learninstruc.2026.102482.

[23] A. Kozanitis, L. Nenciovici, Effect of active learning versus traditional lecturing on the learning achievement of college students in humanities and social sciences: a meta-analysis, Higher Education 86 (6) (2022) 1377–1394. doi:10.1007/s10734-022-00977-8.

[24] C. Dong, Y. Li, H. Gong, M. Chen, J. Li, Y. Shen, M. Yang, A survey of natural language generation, ACM Computing Surveys 55 (8) (2022) 1–38. doi:10.1145/3554727.

[25] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, P. Fung, Survey of hallucination in natural language generation, ACM Computing Surveys 55 (12) (2023) 1–38. doi:10.1145/3571730.

[26] W. Yu, C. Zhu, Z. Li, Z. Hu, Q. Wang, H. Ji, M. Jiang, A survey of knowledge-enhanced text generation, ACM Computing Surveys 54 (11s) (2022) 1–38. doi:10.1145/3512467.

[27] S. Geng, M. Josifoski, M. Peyrard, R. West, Grammar-constrained decoding for structured NLP tasks without finetuning, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 2023, pp. 10932–10952. doi:10.18653/v1/2023.emnlp-main.674.

[28] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, RAGAS: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, St. Julian's, Malta, 2024, pp. 150–158. doi:10.18653/v1/2024.eacl-demo.16.

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, Vol. 36, 2023. doi:10.52202/075280-2020.

[30] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, C. Zhu, G-Eval: NLG evaluation using GPT-4 with better human alignment, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 2023, pp. 2511–2530. doi:10.18653/v1/2023.emnlp-main.153.

[31] Learning Commons, Knowledge graph and evaluators: open infrastructure for grounding AI educational tools in curricula, https://

chanzuckerberg.com/education/ai, Chan Zuckerberg Initiative; accessed 2 November 2026 (2026).

[32] J. Sweller, An integrated human cognitive architecture, Educational Psychology Review 37 (4) (2025). doi:10.1007/s10648-025-10089-1.

[33] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837. doi:10.52202/068431-1800.

[34] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations, 2023. doi:10.48550/arXiv.2210.03629.